第三章 栈，队列和数组

2022年9月30日 星期五 12:52

栈

逻辑结构知识点

栈	• 只允许在一端进行插入或删除操作的线性表
栈顶	• 线性表允许进行插入删除的那一段
栈底	• 固定的，不允许进行插入删除的那一段
操作特性	• 先进后出
数学性质	• n个不同元素进栈，出栈元素的不同排列的个数为 $\frac{1}{n+1}C_{2n}^{n}$（卡特兰数）

存储结构知识点

1、栈的顺序存储结构

顺序栈的定义	• 采用顺序存储的栈 • 利用一组地址连续的存储单元存放自栈底到栈顶的数据元素

顺序栈的实现	`#define Maxsize 50`
	`typedef struct`
	`{`
	`  Elemtype data[Maxsize]; //存放栈中元素`
	`  int top; //栈顶指针`
	`} SqStack;`

栈顶指针	S.top、初始时设置S.top=-1；栈顶元素：S.data[S.top]		
进栈操作	栈不满时，栈顶指针先加1，再送值到栈顶元素	出栈操作	栈非空时，先找栈顶元素值，再将栈顶指针减1
栈空条件	S.top==-1	栈满条件	S.top==Maxsize-1；栈长：S.top+1

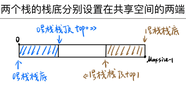
顺序栈基本运算代码

初始化	`void InitStack(SqStack &S)`
	`{`
	`  S.top = -1;`
	`}`
		判栈空	`bool StackEmpty(SqStack S)`
			`{`
			`  if (S.top == -1)//栈空`
			`  return true;`
			`  else`
			`  return false;`
			`}`

进栈	`bool Push(SqStack &S, ElemType x)`
	`{`
	`  if (S.top == Maxsize - 1) //栈满`
	`  return false;`
	`  S.data[++S.top] = x return ture;`
	`}`
		出栈	`bool Pop(SqStack &S, ElemType x)`
			`{`
			`  if (S.top == -1) //栈空`
			`  return false;`
			`  S.data[S.top--] = x return ture;`
			`}`

读栈顶元素	`bool GetTop(SqStack S, ElemType x)`
	`{`
	`  if (S.top == -1)//栈空`
	`  return false;`
	`  x = S.data[S.top] return ture;`
	`}`		

共享栈

定义	• 利用栈底位置相对不变的特性，可让两个顺序栈共享一个一维数组空间 • 将两个栈的栈底分别设置在共享空间的两端，两个栈顶向共享空间的中间延伸
特点	 ① top0 = -1时，0号栈为空 ； top1 = Maxsize时 1号栈为空 ② top1 - top0 = 1时，栈满 ③ 0号栈进栈，top0先加1，再赋值 1号栈进栈，top1先减1，再赋值
目的	• 更有效地利用存储空间 • 两个栈的空间相互调节，只有在整个存储空间都被占满时才发生上溢

2、栈的链式存储结构

概念	• 采用链式存储的栈 Lhead → [ ] → [ ] → … → [^] （栈顶、栈底）
优点	• 便于多个栈共享存储空间和提高其效率，且不存在栈满上溢的情况
特点	• 通常采用单链表实现，并规定所有操作都是在单链表的表头进行的 • 这里规定链栈没有头结点，Lhead指向栈顶元素

链式栈的实现	`typedef struct Linknode`
	`{`
	`  Elemtype data;  //存放栈中元素`
	`  struct Linknode *next; //栈顶指针`
	`} * LiStack;`

栈的运算/操作

InitStack(&S)	初始化栈	• 初始化一个空栈S
StackEmpty(S)	判断栈是否为空	• 空则返回True
Push(&S,x)	进栈	• 若S未满，则将x加入使之成为新栈顶
Pop(&S,&x)	出栈	• 若S非空，则弹出栈顶元素，并用x返回
GetTop(S,&x)	读栈顶元素	• 若栈非空，则用x返回栈顶元素
DestroyStack(&S)	销毁栈	• 销毁并释放栈S占用的存储空间

栈的应用

括号匹配	• 初始设置一个空栈，顺序读入括号 • 若是右括号，置于栈顶 • 若是左括号，压入栈中 • 算法结束时，栈为空，否则括号序列不匹配(具体例子见下面考点3的例题)
表达式求值	• 考点1: 后缀表达式和正常表达式的相互转换 • 考点2: 涉及到编译原理的求值过程 步骤 从左到右遍历 ①数 → 压入 ②操作符 → 取两数计算后压入 考点3: 中缀到后缀表达式转换的过程(见王道P98) 12.【2014-统考真题】假设栈初始为空，将中缀表达式 a/b+(c*d-e*f)/g 转换为等价的后缀表达式的过程中，当扫描到 f 时，栈中的元素依次是(D)。 A. +(*- B. +(-* C. /+(*.* D. /+-* A+B*(C-D)-E/F → 后缀表达式 → ABCD-*+EF/- → 编译原理求值过程 将中缀转后缀步骤 ab/cd*ef*-g/+ ①表达不规范，括号匹配表达一致方法. ②正确转换出后缀表达式即可 ③注意) 直接消掉 (… 风中飘小
递归	• 在递归调用的过程中，系统为每一层的返回点、局部变量、传入实参等开辟了递归工作栈来进行数据存储 • 递归次数过多容易造成栈溢出等 • 将递归算法转换为非递归算法，通常需要借助栈来实现这种转换,消除递归并不一定需要栈 • 效率不高: 原因是递归调用过程中包含很多重复的计算 • 代码简单，容易理解

1	A	push	A		
2	B	push	AB		
3	C	push	ABC		
4	D	push	ABCD		
5	-	C,D pop, C-D=B1, B1 push	ABB1		
6	*	B,B1 pop, B*B1=B2, B2 push	AB2		
7	+	A,B2 pop, A+B2=B3, B3 push	B3		
8	E	push	B3E		
9	F	push	B3EF		
10	/	E,F pop, E/F=B4, B4 push	B3B4		
11	-	B3,B4 pop, B3-B4=B5, B5 push	B5		

队列

逻辑结构知识点

队列	操作受限的线性表
队头	允许删除的一端
队尾	允许插入的一端
操作特性	先进先出

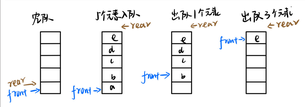
存储结构知识点

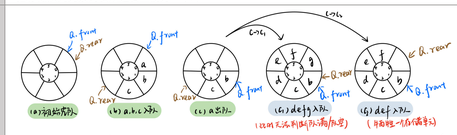
1、队列的顺序存储结构

顺序队列的定义	• 分配一块连续的存储单元存放队列中的元素
	• 设两个指针
	  ○ 队头指针front指向队头元素
	  ○ 队尾指针rear指向队尾元素的下一个位置

顺序队列的实现	`#define MaxSize 50`
	`typedef struct`
	`{`
	`  ElemType data[MaxSize];`
	`  int front, rear;`
	`} SqQueue;`

初始状态	Q.front==Q.rear==0	队满操作	• Q.rear==MaxSize不能作为队列满的条件 • 只有一个元素仍满足该条件（假溢出）
进队操作	队不满时，先送值到队尾元素，再将队尾指针加1	出队操作	队不空时，先取队头元素值，再将队头指针加1



循环队列的定义	• 把存储队列元素的表从逻辑上视为一个环
循环队列的实现 (以下述方法一为例)	 ①初始：Q.front = Q.rear = 0 ②入队，队尾加1：Q.rear = (Q.rear+1)%MaxSize ③出队，队头加1：Q.front = (Q.front+1)%MaxSize ④队列长度：(Q.rear + MaxSize - Q.front)%MaxSize ⑤队空条件：Q.front == Q.rear ⑥队满条件：(Q.rear+1)%MaxSize == Q.front

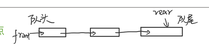
如何区分循环队列队空还是队满	[常用方法，示例图如上↑]	方法二	方法三
定义	牺牲一个单元来区分队空和队满 入队时少用一个队列单元 约定以"队头指针在队尾指针的下一位置作为队满的标志	类型中增设表示元素个数的数据成员	类型中增设tag数据成员，以区分是队满还是队空
队空条件	Q.front==Q.rear	Q.size==0	tag=0且因删除导致Q.front==Q.rear
队满条件	(Q.rear+1)%MaxSize==Q.front	Q.size==MaxSize	tag=0且因插入导致Q.front==Q.rear
元素个数	(Q.rear-Q.front+MaxSize)%MaxSize		

循环队列的操作代码

初始化	`void InitQueue(SqQueue &Q)`
	`{`
	`  Q.rear = Q.front = 0;`
	`}`
		判队空	`bool isEmpty(SqQueue Q)`
			`{`
			`  if (Q.rear == Q.front)`
			`  return true;`
			`  else`
			`  return false;`
			`}`

入队	`bool EnEmpty(SqQueue &Q, ElemType x)`
	`{`
	`  if ((Q.rear + 1) % MaxSize == Q.front)`
	`  return false;`
	`  Q.data[Q.rear] = x;`
	`  Q.rear = (Q.rear + 1) % MaxSize;`
	`  return true;`
	`}`
		出队	`bool DeEmpty(SqQueue &Q, ElemType &x)`
			`{`
			`  if (Q.rear == Q.front)`
			`  return false;`
			`  x = Q.data[Q.front];`
			`  Q.front = (Q.front + 1) % MaxSize;`
			`  return true;`
			`}`

2、队列的链式存储结构

链式队列的定义	• 实质上是一个同时带有队头指针和队尾指针的单链表 • 头指针指向队头节点，尾指针指向队尾节点，即单链表的最后一个节点 • 删除操作时, 通常仅需要修改头指针 • 当队列只有一个元素时, 删除后队列为空, 修改尾指针为read=front 

链式队列的实现	`typedef struct LinkNode`
	`{`
	`  ElemType data;`
	`  struct LinkNode *next;`
	`} LinkNode;`
	`typedef struct`
	`{`
	`  LinkNode *front, *rear;`
	`} LinkNode;`

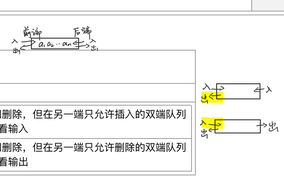
链式队列的操作代码

初始化	`void InitQueue(LinkQueue &Q)`
	`{`
	`  Q.front = Q.rear = (LinkNode *)`
	`malloc(sizeof(LinkNode));`
	`  Q.front->next = NULL;`
	`}`
		判队空	`bool isEmpty(LinkQueue Q)`
			`{`
			`  if (Q.rear == Q.front)`
			`  return true;`
			`  else`
			`  return false;`
			`}`

入队	`bool EnEmpty(LinkQueue &Q, ElemType x)`
	`{`
	`  LinkNode *s = (LinkNode *)malloc(sizeof(LinkNode));`
	`  s->data = x;`
	`  s->next = NULL;`
	`  Q.rear->next = s;`
	`  Q.rear = s;`
	`}`
		出队	`bool DeEmpty(LinkQueue &Q, ElemType &x)`
			`{`
			`  if (Q.rear == Q.front)`
			`  return false;`
			`  LinkNode *p = Q.front->next;`
			`  x = p->data;`
			`  Q.front->next = p->next`
			`  if (Q.rear == p)`
			`  Q.rear = Q.front;`
			`  free(p);`
			`  return true;`
			`}`

3、双端队列

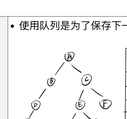
定义	• 允许两端都可以进行入队和出队操作的队列 • 将队列的两端分别称为前端和后端
特点	• 其元素的逻辑结构仍是线性结构
分类	输出受限的双端队列：• 允许在一段进行插入和删除，但在另一端只允许插入的双端队列 • 做题的时候输出固定，看输入 输入受限的双端队列：• 允许在一段进行插入和删除，但在另一端只允许删除的双端队列 • 做题的时候输入固定，看输出



队列的运算/操作

InitQueue(&Q)	初始化队列	• 构造一个空队列Q
QueueEmpty(Q)	判队列为空	• 空则返回True
EnQueue(&Q,x)	入队	• 若队列未满，将x加入，使之成为新的队尾
DeQueue(&Q,&x)	出队	• 若队列非空，删除表头元素，并用x返回
GetHead(Q,&x)	读队头元素	• 若队列非空，则将队头元素赋值给x

队列的应用

层次遍历	• 使用队列是为了保存下一步的处理顺序  A B C D E F G H I J 1 A入 A A 2 A出, B,C入 BC B 3 B出, D入 CD BD 4 C出, E,F入 DEF BDE 5 D出, G入 EF BDEG 6 E出, H,I入 FGHI BDEGH 7 F出 GHI BDEGHF 8 G,H,I出 BDEGHFGHI
在计算机系统中的应用	1. 解决主机与外部设备之间速度不匹配的问题（如打印机与主机，设置一个打印数据缓冲区） 2. 解决由多用户引起的资源竞争问题（如CPU资源的竞争） 3. 页面替换算法(FIFO算法)

数组和特殊矩阵

本部分重点	• 如何将矩阵更有效地存储在内存中，并能方便地提取矩阵中的元素
数组的定义	• 由n个相同类型的数据元素构成的有限序列
数组的特点	• 数组一旦被定义，其维数和维界就不再改变 • 除结构的初始化和销毁外，数组只会有存取元素和修改元素的操作
数组的数据结构	一个数组的所有元素在内存中占用一段连续的存储空间 对于多维数组，有两种映射方法 按行优先：先行后列，先存储行号较小的元素，行号相等先存储列号较小的元素 按列优先：先列后行，先存储列号较小的元素，列号相等先存储行号较小的元素
特殊矩阵相关概念	压缩存储：为多个值相同的元素只分配一个空间，对零元素不分配存储空间，其目的是节省存储空间 特殊矩阵：• 具有许多相同矩阵元素或者零元素，并且这些相同矩阵元素或零元素的分布有一定规律性的矩阵 • 常见的特殊矩阵有对称矩阵、三角矩阵（第一行和最后一行两个元素，其余行3个元素）、对角矩阵 • 常考某个值对应的行和列(用特殊法代指就好，不用记那么多公式，可以用线性代数的知识解决)
特殊矩阵的压缩存储方法	• 找到特殊矩阵中值相同的矩阵元素的分布规律 • 把这些呈现规律性分布的、值相同的多个矩阵元素压缩存储到一个存储空间中
稀疏矩阵的定义	• 非0元素非常少的矩阵
稀疏矩阵的特点	• 稀疏矩阵压缩存储后便失去了随机存取特性 • 可以使用三元组表和十字链表法压缩存储稀疏矩阵